

BiblioInstituto

Módulo Odoo 18 — Sistema de gestión de biblioteca escolar

Memoria técnica del proyecto

Alumno: Javier Gisbert Talboom

Ciclo: 2.º DAM — Desarrollo de Aplicaciones Multiplataforma

Módulo: SGE — Sistemas de Gestión Empresarial

Tecnologías: Odoo 18 · Python 3 · PostgreSQL · XML · QWeb · Docker

Fecha: Junio 2026

ÍNDICE

1. Introducción y objetivos
2. Infraestructura: Docker y arquitectura
3. Estructura del módulo Odoo
4. Modelos de datos y decisiones de diseño
5. Vistas: tipos y justificación
6. Sistema de seguridad en tres capas
7. Informes QWeb
8. Ampliaciones opcionales
9. Datos de demostración
10. Usuarios del sistema

1 Introducción y objetivos

BiblioInstituto es un módulo desarrollado sobre la plataforma Odoo 18 que implementa un sistema completo de gestión de biblioteca para centros educativos. El proyecto se ha realizado como trabajo de recuperación de la asignatura SGE (Sistemas de Gestión Empresarial) del ciclo formativo de 2.º DAM.

Alcance funcional

Catálogo

Gestión de libros, autores y editoriales con todos sus metadatos, portadas y control de existencias.

Préstamos

Ciclo completo: alta, renovación, devolución, control de vencimientos y generación de comprobante PDF.

Usuarios

Extensión del modelo de contactos de Odoo con datos específicos de biblioteca: carnet, tipo de usuario y límite de préstamos.

Objetivos técnicos cubiertos

Objetivo	Implementación	Requisito
Modelos ORM con relaciones	5 modelos + extensión de res.partner	Obligatorio
Campos calculados con @api.depends	available_copies, state, late_days, name	Obligatorio
Validaciones Python	Límite de préstamos, número ejemplar, máximo negativo	Obligatorio
Vistas list, form, kanban, search	Todas las entidades tienen sus vistas completas	Obligatorio
Sistema de seguridad	2 grupos + CSV de accesos + reglas de registro	Obligatorio
Informe QWeb PDF	Comprobante de préstamo desde formulario	Obligatorio
Vista Calendar	Calendario mensual de vencimientos	Ampliación +nota
Wizard de préstamos múltiples	Asistente modal con TransientModel	Ampliación +nota

2 Infraestructura: Docker y arquitectura

El entorno de desarrollo utiliza Docker Compose para levantar dos servicios aislados: el servidor Odoo y la base de datos PostgreSQL. Esta elección elimina la necesidad de instalar dependencias localmente y garantiza que el entorno sea reproducible en cualquier máquina.

Servicios definidos en docker-compose.yml

Servicio	Imagen	Puerto
web (Odoo)	odoo:18	8069 → host
db (PostgreSQL)	postgres:15	5432 (interno)

Volúmenes persistentes

Volumen	Contenido
postgres_data	Toda la base de datos
odoo_filestore	Imágenes, adjuntos
./volumesOdoo/addons	Código del módulo (montado en vivo)

Decisión de diseño: montaje de addons como volumen

El directorio `./volumesOdoo/addons` se monta directamente dentro del contenedor en `/mnt/extra-addons`. Esto significa que cualquier cambio en los archivos Python o XML del módulo es visible inmediatamente en el contenedor sin reconstruir la imagen. Solo se necesita hacer "Actualizar módulo" en la interfaz de Odoo para aplicar los cambios.

Diferencia crítica: restart vs down -v

`docker compose restart` reinicia los contenedores pero conserva todos los volúmenes (datos de BD intactos). `docker compose down -v` elimina también los volúmenes, borrando todos los datos. Durante el desarrollo solo se usa restart o down (sin -v).

3 Estructura del módulo Odoo

```
library_instituto/
├── __manifest__.py          ← "pasaporte" del módulo: nombre, versión, dependencias, archivos a cargar
├── __init__.py              ← importa el paquete models/ para que Python cargue los modelos
│
├── models/                  ← lógica de negocio en Python (clases ORM)
│   ├── __init__.py          ← importa cada fichero .py del directorio
│   ├── library_author.py    ← modelo library.author
│   ├── library_book.py      ← modelo library.book (con campos calculados)
│   ├── library_publisher.py ← modelo library.publisher
│   ├── library_loan.py      ← modelo library.loan (lógica principal)
│   ├── library_loan_wizard.py  ← TransientModel para préstamos múltiples
│   └── res_partner.py       ← extensión de res.partner con _inherit
│
├── views/                   ← definición de la interfaz en XML
│   ├── library_author.xml
│   ├── library_book.xml     ← incluye vistas list, form, kanban y search
│   ├── library_publisher.xml
│   ├── library_loan.xml     ← incluye vista calendar (ampliación)
│   ├── library_loan_wizard.xml  ← formulario modal del wizard
│   ├── res_partner_library.xml  ← kanban de usuarios con foto y badges
│   └── library_menus.xml    ← árbol de menús con control de visibilidad por grupo
│
├── security/                ← todo lo relacionado con permisos y acceso
│   ├── security.xml         ← define los dos grupos del módulo
│   ├── ir.model.access.csv  ← permisos CRUD por grupo y modelo
│   └── library_security_rules.xml  ← filtro de registros (préstamos propios)
│
├── reports/                 ← plantillas QWeb para generación de PDF
│   └── report_loan.xml      ← comprobante de préstamo
│
└── data/                    ← datos que se cargan al instalar el módulo
    └── library_demo_clean.xml  ← 3 editoriales, 8 autores, 10 socios, 20 libros, préstamos
```

El archivo __manifest__.py y el orden de carga

El manifest es el primer archivo que Odoo lee al instalar o actualizar un módulo. La clave `"data"` contiene la lista ordenada de archivos XML/CSV que se cargarán. **El orden es determinante**: si un archivo XML referencia un elemento definido en otro, ese otro debe ir primero.

```
"data": [
    "security/security.xml",            # 1º: grupos (otros archivos los
referencian)
    "security/ir.model.access.csv",      # 2º: permisos (necesita los grupos
ya creados)
    "security/library_security_rules.xml", # 3º: reglas de registro
(necesita grupos y modelos)
    "views/library_author.xml",          # vistas de entidades simples
    "views/library_book.xml",
    "views/library_publisher.xml",
    "data/library_demo_clean.xml",       # datos demo (necesita que los
modelos existan)
    "views/library_loan.xml",
    "views/library_loan_wizard.xml",     # wizard antes que el menú que lo
referencia
    "views/res_partner_library.xml",
    "views/library_menus.xml",           # menús al final: referencian
todas las acciones anteriores
    "reports/report_loan.xml"
],
```

4 Modelos de datos y decisiones de diseño

> **Principio ORM de Odoo**
>
> Cada clase Python que hereda de `models.Model` se convierte automáticamente en una tabla de PostgreSQL. Los atributos de clase con `fields.*` generan columnas. No se escribe SQL: el ORM (Object Relational Mapper) traduce las operaciones Python a consultas SQL. Esto abstrae la capa de datos y permite cambiar de motor de BD sin modificar el código.

4.1 library.author y library.publisher — Entidades de catálogo

Modelos de apoyo al catálogo. Tienen estructura sencilla y se relacionan con los libros mediante Many2one/Many2many. Se les aplica `_sql_constraints` para garantizar unicidad de nombre a nivel de base de datos, que es más eficiente que una validación Python.

4.2 library.book — Libro del catálogo

Decisión: _rec_name = "title"

Por convención, Odoo busca el campo "nombre visible" de un modelo en un campo llamado `name`. Como aquí el campo se llama `title`, es necesario indicarlo explícitamente. Sin esto, los desplegables Many2one mostrarían el ID técnico del registro en lugar del título del libro.

Decisión: _sql_constraints para ISBN

El ISBN es un identificador único por definición. Se implementa como `UNIQUE(isbn)` a nivel SQL porque es más robusto: la restricción la impone PostgreSQL directamente, incluso si se insertan datos desde fuera del ORM de Odoo.

Campos calculados: active_loans_count y available_copies

```
@api.depends("total_copies", "loan_ids.state")
def _compute_available_copies(self):
    # Cuenta préstamos activos o vencidos (excluye devueltos) y calcula
ejemplares libres
    for book in self:
        active_loans = len(book.loan_ids.filtered(lambda loan: loan.state in
["activo", "vencido"]))
        book.active_loans_count = active_loans
        book.available_copies   = max((book.total_copies or 0) -
active_loans, 0)
```

¿Por qué loan_ids.state con notación de punto en @api.depends?

La notación `"loan_ids.state"` le indica a Odoo que debe recalcular este campo no solo cuando se añada o elimine un préstamo del libro, sino también cuando cambie el estado (`state`) de cualquiera de esos préstamos. Sin el `.state`, el campo no se recalcularía al devolver un libro, ya que la devolución cambia el estado del préstamo pero no crea ni borra el registro.

¿Por qué ambos campos en un mismo método?

Ambos campos necesitan exactamente la misma información: el conteo de préstamos activos. Calcularlos en métodos separados implicaría recorrer y filtrar la lista de préstamos dos veces por cada libro. Un único método que calcula ambos a la vez es significativamente más eficiente cuando hay muchos libros.

4.3 library.loan — Préstamo (modelo central)

Es el modelo con mayor lógica de negocio. Gestiona el ciclo de vida completo de un préstamo.

Campo name con store=False

```
name = fields.Char(string="Referencia", compute="_compute_name",
store=False)

@api.depends("user_id", "book_id")
def _compute_name(self):
    for loan in self:
        loan.name = f"{loan.user_id.display_name or '?'} —
{loan.book_id.title or '?'}"
```

Justificación técnica de store=False

La tabla `library_loan` fue creada en la instalación inicial del módulo sin esta columna. Añadir un campo calculado con `store=True` requeriría que Odoo ejecute un `ALTER TABLE ADD COLUMN` en PostgreSQL al actualizar el módulo, lo que puede fallar si la tabla ya existe con datos. Con `store=False` el valor se calcula en memoria Python cada vez que se necesita, sin tocar el esquema de la base de datos. La razón de necesitar este campo es que la vista Calendar de Odoo utiliza el campo `name` del modelo como etiqueta de cada evento; sin él, los eventos mostrarían el identificador técnico `library.loan,N`.

Máquina de estados automática

Estado	Condición
activo	No hay fecha de devolución real y la fecha límite no ha pasado
vencido	No hay fecha de devolución real y la fecha límite ya pasó
devuelto	Existe fecha de devolución real

El estado se recalcula automáticamente mediante `@api.depends("real_return_date", "expected_return_date")`. Cuando el bibliotecario pulsa "Devolver", el método `action_return()` asigna la fecha real; esto dispara el recálculo del estado sin necesidad de asignarlo manualmente.

Constraint de límite de préstamos simultáneos

```
@api.constrains("user_id", "state")
def _check_loan_limit(self):
    active_states = ["activo", "vencido"]
    for loan in self:
        if not loan.user_id or loan.state not in active_states:
            continue
        active_loans = loan.user_id.library_loan_ids.filtered(
            lambda item: item.state in active_states and item.id != loan.id
        )
        if len(active_loans) >= loan.user_id.library_max_loans:
            raise ValidationError(...)
```

> **item.id != loan.id — exclusión del préstamo actual**
>
> Cuando se crea o modifica un préstamo, el constraint se ejecuta sobre ese mismo registro que ya fue insertado en la base de datos. Si no se excluyera, el sistema lo contaría como uno de los préstamos activos ya existentes, generando un falso positivo que impediría crear préstamos incluso cuando el usuario está dentro del límite.

4.4 res.partner — Extensión del modelo de contactos

```
class ResPartner(models.Model):
    _inherit = "res.partner"   # _inherit, no _name: amplía la tabla
existente
```

_inherit vs _name: diferencia fundamental

`_name = "library.book"` crea un modelo completamente nuevo con su propia tabla en PostgreSQL.

`_inherit = "res.partner"` amplía un modelo ya existente añadiendo nuevas columnas a su tabla. No se crea ninguna tabla nueva. Es la técnica estándar de Odoo para extender modelos del núcleo sin modificar el código fuente de la plataforma, respetando el principio Open/Closed de la programación orientada a objetos. Todos los socios, clientes, proveedores y empleados de Odoo son `res.partner`; extender este modelo permite que los usuarios de biblioteca sean el mismo concepto que los demás contactos del sistema.

4.5 library.loan.wizard — Wizard de préstamos múltiples Ampliación

```
class LibraryLoanWizard(models.TransientModel):
    _name = "library.loan.wizard"
    user_id  = fields.Many2one("res.partner", required=True)
    line_ids = fields.One2many("library.loan.wizard.line", "wizard_id")

    def action_confirm(self):
        # Crea un préstamo real por cada línea de la tabla al pulsar
"Confirmar"
        today = fields.Date.context_today(self)
        for line in self.line_ids:
            self.env["library.loan"].create({
                "user_id": self.user_id.id,
                "book_id": line.book_id.id,
                "loan_date": today,
                "expected_return_date": today + timedelta(days=15),
            })
```

TransientModel vs Model

`models.TransientModel` crea registros temporales en PostgreSQL que Odoo elimina automáticamente mediante un job periódico (normalmente cada hora). Es adecuado para los wizards porque los datos del asistente solo son relevantes durante la interacción del usuario; una vez confirmado o cancelado no tienen valor permanente y no deben ocupar espacio en la base de datos indefinidamente.

self.env["library.loan"]

`self.env` es el entorno de ejecución de Odoo, que contiene el usuario activo, el contexto de zona horaria y el acceso al registro de todos los modelos. `self.env["library.loan"]` devuelve la clase Python del modelo indicado, permitiendo llamar a sus métodos estándar como `.create()`, `.search()` o `.write()` desde cualquier otro modelo.

Diseño en dos modelos (cabecera + líneas)

La estructura "cabecera - líneas" es el patrón estándar de Odoo para formularios con tablas dinámicas (como una factura o un albarán). La cabecera (`library.loan.wizard`) guarda el usuario destinatario. Las líneas (`library.loan.wizard.line`) son los libros individuales y se almacenan como registros independientes vinculados mediante Many2one. Esto permite añadir y eliminar filas dinámicamente en el formulario mediante el campo One2many, algo que no sería posible con campos sueltos en el mismo modelo.

5 Vistas: tipos y justificación

Una vista en Odoo es un registro XML guardado en la tabla `ir.ui.view` de PostgreSQL que define cómo se renderiza un modelo en la interfaz web. Odoo selecciona la vista adecuada según el contexto.

Tipo de vista	Uso en el proyecto	Justificación
list	Todos los modelos	Vista por defecto para explorar colecciones de datos. Permite ordenar, filtrar y selección múltiple.
form	Todos los modelos	Vista detallada de un registro individual. Contiene los botones de acción (Renovar, Devolver) y el statusbar.
kanban	Libros y usuarios	Libros: presentación visual con portadas. Usuarios: presentación con foto y badge de tipo. Mejora la experiencia del bibliotecario.
calendar	Préstamos	Ampliación opcional. Permite visualizar vencimientos distribuidos en el mes, detectando días con alta carga de devoluciones.
search	Libros y préstamos	Define los campos de búsqueda libre y los filtros rápidos por estado y género.

Vista Form: estructura header + sheet

```
<form>
  <header>
    <!-- Botones de acción y statusbar van aquí -->
    <button name="action_renew" type="object" invisible="state == 'devuelto'
or renewed"/>
    <field name="state" widget="statusbar"
statusbar_visible="activo,vencido,devuelto"/>
  </header>
  <sheet>
    <!-- Campos de datos del registro -->
  </sheet>
</form>
```

El atributo `invisible="state == 'devuelto' or renewed"` es una expresión Python evaluada en el cliente. Oculta el botón "Renovar" cuando el préstamo ya está devuelto o ya fue renovado, guiando al usuario sin necesidad de mensajes de error.

Vista Kanban: diferencia entre servidor y cliente

¿Por qué la URL de las imágenes usa + y no %s?

Las plantillas Kanban se renderizan en el **navegador del usuario** mediante JavaScript, no en el servidor Python. Por eso la concatenación de la URL de imagen debe hacerse con el operador `+` de JavaScript. Si se usara la sintaxis `%s` de Python, la plantilla no funcionaría porque Python no la procesa; solo JavaScript la interpreta en ese contexto.

`t-att-src="'/web/image/library.book/' + record.id.raw_value + '/cover_image'"`
`raw_value` extrae el valor numérico del ID porque en kanban los campos Many2one devuelven un array `[id, "nombre"]`.

Vista Calendar Ampliación

```
<calendar
    date_start="expected_return_date"
    mode="month"
    color="state">
  <field name="user_id"/>
  <field name="book_id"/>
</calendar>
```

Atributo	Efecto
`date_start="expected_return_date"`	Cada préstamo aparece en el día de su fecha de devolución prevista, no en la de alta.
`mode="month"`	Se abre en vista mensual por defecto. El usuario puede cambiar a semana o día.
`color="state"`	Odoo asigna automáticamente un color distinto a cada valor posible del campo state.

Vista Wizard (modal) Ampliación

La acción que abre el wizard define `target="new"`, que hace que el formulario se abra como ventana modal superpuesta a la pantalla actual en lugar de navegar a una nueva página. El botón "Cancelar" usa `special="cancel"`, un atributo exclusivo de wizards modales que cierra la ventana sin ejecutar ningún método Python ni guardar datos.

Menús y control de visibilidad

El atributo `groups` en un `menuitem` hace que Odoo renderice ese ítem solo para usuarios pertenecientes al grupo indicado. Los menús "Usuarios biblioteca" y "Préstamo múltiple" están restringidos a `group_library_librarian` por razones de privacidad (un alumno no debe ver los datos de carnet de otros usuarios) y de lógica de negocio (crear préstamos es una tarea del personal de biblioteca).

6 Sistema de seguridad en tres capas

Odoo implementa la seguridad en tres niveles independientes que actúan conjuntamente. El orden de carga en el manifest respeta esta jerarquía: primero los grupos, luego los accesos que los referencian, finalmente las reglas que usan tanto grupos como modelos.

Capa 1: Grupos (security.xml)

Define los roles de usuario. Un usuario puede pertenecer a varios grupos. Los grupos son el elemento base al que se refieren las capas 2 y 3.

Capa 2: Accesos a modelos (ir.model.access.csv)

Define qué operaciones CRUD puede hacer cada grupo sobre cada modelo. Es el filtro de tipo de operación.

Capa 3: Reglas de registro (library_security_rules.xml)

Filtros a nivel de fila. Determinan qué registros concretos puede ver o modificar cada grupo, más allá del acceso al modelo.

Los dos grupos del módulo

Grupo	ID interno	Perfil de usuario
Bibliotecario	group_library_librarian	Personal de biblioteca: acceso CRUD completo a todos los modelos del módulo.
Usuario biblioteca	group_library_user	Alumnos y docentes: solo lectura de libros, autores y editoriales. Solo lectura de sus propios préstamos.

Tabla de permisos CRUD (ir.model.access.csv)

Formato del CSV: `id, name, model_id:id, group_id:id, perm_read, perm_write, perm_create, perm_unlink`

Modelo	Admin (group_system)	Bibliotecario	Usuario
library.book / author / publisher	CRUD	CRUD	R (solo lectura)
library.loan	CRUD	CRUD	R (filtrado por regla)
library.loan.wizard	CRUD	CRUD	Sin acceso
library.loan.wizard.line	CRUD	CRUD	Sin acceso

¿Por qué el wizard tiene permisos para base.group_system (admin) además de para el bibliotecario?

En Odoo Community Edition, el usuario administrador (`base.group_system`) no es un superusuario que bypasea todos los controles de acceso. Necesita permisos explícitos en el CSV para cada modelo, igual que cualquier otro grupo. Si el wizard solo tuviera permisos para `group_library_librarian` y el administrador no perteneciera a ese grupo, Odoo ocultaría el menú del wizard para el administrador sin mostrar ningún error visible, lo que generaría confusión durante las pruebas y el uso del sistema.

Regla de registro — Filtro de préstamos propios

```
<!-- library_security_rules.xml -->
<record id="library_loan_rule_own_records" model="ir.rule">
  <field name="model_id" ref="model_library_loan"/>
  <field name="groups" eval="[(4,
ref('library_instituto.group_library_user'))]"/>
  <field name="domain_force">[('user_id', '=', user.partner_id.id)]</field>
</record>
```

`domain_force` es un filtro que Odoo añade automáticamente a todas las consultas SQL del modelo para ese grupo. `user.partner_id.id` es el ID del partner del usuario activo en ese momento. El resultado es que un alumno que consulta la lista de préstamos solo recibe los suyos propios, sin necesidad de ningún código Python adicional.

> **¿Por qué no hay regla equivalente para el bibliotecario?**
>
> Las reglas de registro en Odoo son restrictivas: se aplican únicamente al grupo indicado. Si un grupo no tiene ninguna regla para un modelo, ve todos los registros. El bibliotecario no tiene regla, por tanto ve todos los préstamos. El usuario de biblioteca sí tiene regla, por tanto solo ve los suyos. Este diseño es deliberado: añadir una regla solo donde se necesita restricción, no en todos los grupos.

7 Informes QWeb

QWeb es el motor de plantillas de Odoo. Genera HTML dinámico que el servidor convierte a PDF mediante `wkhtmltopdf`, una herramienta instalada en el contenedor Docker. La plantilla es XML con atributos especiales prefijados con `t-`.

Estructura del informe (report_loan.xml)

El archivo combina dos elementos en uno:

```
<!-- Elemento 1: acción que registra el informe en el sistema -->
<record id="action_report_loan" model="ir.actions.report">
  <field name="model">library.loan</field>
  <field name="report_type">qweb-pdf</field>
  <field name="report_name">library_instituto.report_loan_template</field>
  <!-- binding_model_id: añade automáticamente el informe al menú Imprimir
       de cualquier formulario del modelo, sin necesidad de botones manuales
-->
  <field name="binding_model_id" ref="model_library_loan"/>
  <field name="binding_type">report</field>
</record>

<!-- Elemento 2: plantilla visual del documento PDF -->
<template id="report_loan_template">
  <t t-call="web.html_container">
    <t t-foreach="docs" t-as="loan">        <!-- itera sobre los préstamos
seleccionados -->
      <t t-call="web.external_layout">      <!-- cabecera/pie con datos de
la empresa -->
        <div class="page">
          <t t-esc="loan.user_id.display_name"/>   <!-- imprime valor
escapando HTML -->
          <t t-esc="loan.book_id.title"/>
          <t t-esc="loan.loan_date"/>
          <t t-esc="loan.expected_return_date"/>
          <!-- Convierte el valor interno 'activo' → etiqueta 'Activo' -->
          <t t-esc="dict(loan.fields_get(['state'])['state']['selection'])
[loan.state]"/>
        </div>
      </t>
    </t>
  </t>
</template>
```

Directivas QWeb utilizadas

Directiva	Función
`t-call="plantilla"`	Incluye otra plantilla, como un `include`. Se usa para reutilizar la estructura HTML base de Odoo (`web.html_container`, `web.external_layout`).
`t-foreach="docs" t-as="loan"`	Bucle sobre la colección de registros. `docs` es la variable estándar que Odoo pasa con los registros seleccionados para el informe.
`t-esc="expresión"`	Evalúa la expresión Python y renderiza el resultado como texto escapado (seguro contra inyección HTML).

Conversión del valor de selection a etiqueta legible

Un campo `Selection` almacena en BD el valor interno (`"activo"`) pero para el PDF se necesita la etiqueta visible (`"Activo"`). La expresión `dict(loan.fields_get(['state'])['state']['selection'])[loan.state]` obtiene el diccionario de opciones del campo y recupera la etiqueta correspondiente al valor actual.

8 Ampliaciones opcionales

Vista Calendar +nota

Añade un tipo de vista al listado de préstamos que los presenta como eventos en un calendario mensual, distribuidos en la fecha de devolución prevista de cada uno.

Valor funcional: el personal puede ver de un vistazo qué días del mes tienen mayor concentración de devoluciones y detectar préstamos vencidos por su color.

Implementación técnica: se añadió la vista `<calendar>` en `views/library_loan.xml` y se incluyó `calendar` en el `view_mode` de la acción. Fue necesario añadir un campo `name` calculado al modelo `library.loan` con `store=False` para que los eventos del calendario mostraran texto legible.

Wizard préstamos múltiples +nota

Asistente emergente (ventana modal) que permite registrar varios préstamos a un mismo usuario en una sola operación.

Valor funcional: cuando un alumno lleva varios libros, el bibliotecario puede registrar todos los préstamos de una vez en lugar de crearlos uno a uno.

Implementación técnica: dos modelos `TransientModel` (cabecera + líneas), formulario con `target="new"` para apertura modal, método `action_confirm()` que itera las líneas y crea los préstamos reales usando `self.env["library.loan"].create()`.

9 Datos de demostración

El archivo `data/library_demo_clean.xml` se carga automáticamente al instalar el módulo, proporcionando datos de ejemplo para poder probar todas las funcionalidades sin necesidad de introducirlos manualmente.

Entidad	Cantidad	Detalle
Editoriales	3	Alfaguara, Planeta, Penguin Random House
Autores	8	Con nombre, país y fecha de nacimiento
Usuarios (res.partner)	10	4 alumnos, 3 docentes, 2 personal de administración, 1 admin. Todos con carnet BIB001–BIB010, email, teléfono y foto.
Libros	20	Con ISBN, género, editorial y número de copias
Préstamos	Varios	En distintos estados: activos, vencidos y devueltos para cubrir todos los casos de prueba

¿Por qué los datos de demo están en "data" y no en "demo" del manifest?

Odoo distingue la sección `"data"` (se carga siempre al instalar) de la sección `"demo"` (solo se carga en instancias creadas con el flag de modo demo). El entorno Docker de este proyecto no fue creado en modo demo, por lo que si los datos estuvieran en la sección `"demo"`, nunca se cargarían. Para garantizar que los datos de ejemplo estén disponibles en esta instancia concreta, se mantienen en la sección `"data"`.

10 Usuarios del sistema

Los siguientes usuarios de Odoo están configurados en la instancia de desarrollo para probar los distintos niveles de acceso del módulo:

Correo electrónico	Contraseña	Rol en Odoo	Grupo biblioteca	Acceso
javigisbert05@gmail.com	`bibliotecainstituto`	**Administrador**	Todos los grupos	Acceso completo al sistema. Puede gestionar usuarios, instalar módulos y acceder al menú Técnico.
fonsi.biblio@biblio.com	`fonsi`	**Usuario interno**	Bibliotecario	CRUD completo en todos los modelos del módulo. Ve todos los préstamos. Accede al wizard y a la lista de usuarios.
benito.biblio@bibio.com	`Benito`	**Usuario interno**	Usuario biblioteca	Solo lectura en libros, autores y editoriales. Ve únicamente sus propios préstamos. No accede al wizard ni a la lista de usuarios.

> **Nota sobre las credenciales**
>
> Las contraseñas indicadas son las configuradas en el entorno de desarrollo local (Docker). No son contraseñas de producción. Para un entorno productivo deben seguirse las recomendaciones de seguridad de Odoo: contraseñas complejas, HTTPS obligatorio y desactivación de la interfaz de administración de base de datos.